

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**COURSE CODE: ITA 0601**

**COURSE NAME: MACHINE LEARNING**

### **COURSE OUTCOME COVERED**

**CO1:** *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

*Assessment Tool Weightage: 40%*

**QUESTIONS: 15**

**Total Marks: 25**

Annexure A

### **APPLICATION BASED QUESTIONS**

#### ***Code of Conduct:***

*I V.Abdul Azeez(192312089) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

#### ***Signature of the student:***

*V.Abdul Azeez*

1. A factory has three boxes containing colored balls. Box A contains 6 red, 4 blue, and 2 green balls; Box B contains 5 red, 7 blue, and 3 green balls; and Box C contains 8 red, 2 blue, and 5 green balls. One box is selected at random, but Box B is twice as likely to be chosen as each of the other boxes. A ball drawn from the chosen box is found to be blue. What is the probability that the selected box was Box C?

The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script named 'main.py' that calculates the probability of selecting Box C given a blue ball was drawn. The script uses Bayes' Theorem to find the probability.

```

1 P_A = 1/4
2 P_B = 1/2
3 P_C = 1/4
4
5 # Probability of drawing a blue ball
6 P_Blue_A = 4/12
7 P_Blue_B = 7/15
8 P_Blue_C = 2/15
9
10 # Total probability of drawing blue
11 P_Blue = (P_Blue_A * P_A) + (P_Blue_B * P_B) + (P_Blue_C * P_C)
12
13 # Bayes Theorem
14 P_C_given_Blue = (P_Blue_C * P_C) / P_Blue
15
16 print("Probability that the selected box was Box C =", round(P_C_given_Blue, 4))

```

The output of the script is displayed in the Output panel:

```

Probability that the selected box was Box C = 0.0952
=== Code Execution Successful ===

```

2. A parking facility has three sections containing cars of different colors. Section A has 5 red, 3 blue, and 2 black cars; Section B has 4 red, 6 blue, and 5 black cars; and Section C has 7 red, 2 blue, and 6 black cars. A section is chosen at random, but Section B is twice as likely to be selected as each of the other sections due to higher usage. A randomly selected car from the chosen section turns out to be blue. What is the probability that the car came from Section C?

```

main.py
1 # Section selection probabilities
2 P_A = 1/4
3 P_B = 1/2
4 P_C = 1/4
5
6 # Probability of selecting a blue car
7 P_Blue_A = 3/10
8 P_Blue_B = 6/15
9 P_Blue_C = 2/15
10
11 # Total probability of blue car
12 P_Blue = (P_Blue_A * P_A) + (P_Blue_B * P_B) + (P_Blue_C * P_C)
13
14 # Bayes Theorem
15 P_C_given_Blue = (P_Blue_C * P_C) / P_Blue
16
17 print("Probability that the car came from Section C =", round(P_C_given_Blue, 4))

```

Output

Probability that the car came from Section C = 0.1081

=== Code Execution Successful ===

3. In a real-world medical image classification task for detecting breast cancer, a deep-learning model integrated with the K-Nearest Neighbors algorithm is applied for binary classification (malignant vs. benign). The dataset consists of 1800 samples per class, with 20% of the data reserved for testing. During evaluation, the model reports 210 True Positives (TP), 190 True Negatives (TN), and 30 False Positives (FP). Based on the given information, determine the missing False Negatives (FN) value, and then calculate the Accuracy, Precision, Sensitivity (Recall), and Specificity of the classifier, explaining its effectiveness in medical diagnosis.

```

main.py
1 # Given values
2 TP = 210
3 TN = 190
4 FP = 30
5
6 # Total samples
7 samples_per_class = 1800
8 total_samples = 3600
9
10 # 20% testing data
11 test_samples = int(total_samples * 0.20)
12
13 # Calculate FN
14 FN = test_samples - (TP + TN + FP)
15
16 # Metrics
17 accuracy = ((TP + TN) / test_samples) * 100
18 precision = (TP / (TP + FP)) * 100
19 recall = (TP / (TP + FN)) * 100
20 specificity = (TN / (TN + FP)) * 100
21
22 print("FN =", FN)
23 print("Accuracy =", round(accuracy, 2), "%")
24 print("Precision =", round(precision, 2), "%")
25 print("Sensitivity (Recall) =", round(recall, 2), "%")
26 print("Specificity =", round(specificity, 2), "%")

```

Output

FN = 290

Accuracy = 55.56 %

Precision = 87.5 %

Sensitivity (Recall) = 42.0 %

Specificity = 86.36 %

=== Code Execution Successful ===

4. Examine the performance of a machine-learning model using its training and validation loss values over multiple epochs

a. Epoch 1: Training Loss = 2.8, Validation Loss = 2.4

- a. Epoch 2: Training Loss = 2.5, Validation Loss = 2.2  
 (c) Epoch 3: Training Loss = 2.2, Validation Loss = 2.0  
 (d) Epoch 4: Training Loss = 2.0, Validation Loss = 2.1  
 (e) Epoch 5: Training Loss = 1.8, Validation Loss = 2.3  
 (f) Epoch 6: Training Loss = 1.6, Validation Loss = 2.6  
 (g) Epoch 7: Training Loss = 1.5, Validation Loss = 2.9  
 (h) Epoch 8: Training Loss = 1.3, Validation Loss = 3.2
- i) Identify the epoch at which the model begins to overfit.  
 ii) Suggest two effective techniques to reduce overfitting in this model.  
 iii) Discuss how overfitting affects the model's generalization ability on unseen data.

The screenshot shows a Python Online Compiler interface. The code in the editor identifies the epoch where overfitting begins by comparing training and validation losses. It also lists techniques to reduce overfitting and discusses its effect on generalization.

```

main.py
1 training_loss = [2.8, 2.5, 2.2, 2.0, 1.8, 1.6, 1.5, 1.3]
2 validation_loss = [2.4, 2.2, 2.0, 2.1, 2.3, 2.6, 2.9, 3.2]
3
4 overfit_epoch = None
5
6 for i in range(1, len(validation_loss)):
7     if validation_loss[i] > validation_loss[i-1]:
8         overfit_epoch = i + 1
9         break
10
11 print("Model starts overfitting at Epoch:", overfit_epoch)
12
13 print("\nTechniques to reduce overfitting:")
14 print("1. Early Stopping")
15 print("2. Dropout / Regularization")
16
17 print("\nEffect:")
18 print("Overfitting reduces generalization ability and causes poor performance on
   unseen data.")
  
```

Output:

```

Model starts overfitting at Epoch: 4

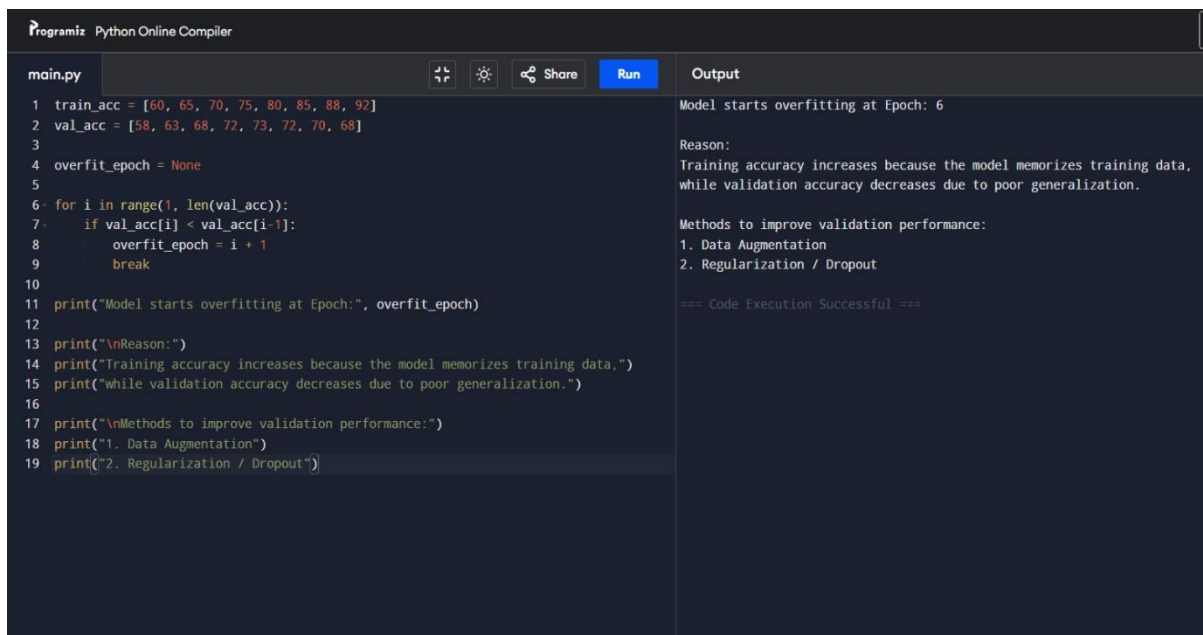
Techniques to reduce overfitting:
1. Early Stopping
2. Dropout / Regularization

Effect:
Overfitting reduces generalization ability and causes poor performance on unseen data.

=== Code Execution Successful ===
  
```

5. Examine the performance of a machine-learning model using training and validation accuracy over multiple epochs
- (a) Epoch 1: Training Accuracy = 60%, Validation Accuracy = 58%  
 (b) Epoch 2: Training Accuracy = 65%, Validation Accuracy = 63%  
 (c) Epoch 3: Training Accuracy = 70%, Validation Accuracy = 68%  
 (d) Epoch 4: Training Accuracy = 75%, Validation Accuracy = 72%  
 (e) Epoch 5: Training Accuracy = 80%, Validation Accuracy = 73%  
 (f) Epoch 6: Training Accuracy = 85%, Validation Accuracy = 72%

- (g) Epoch 7: Training Accuracy = 88%, Validation Accuracy = 70%
- (h) Epoch 8: Training Accuracy = 92%, Validation Accuracy = 68%
- i) Identify the epoch at which the model starts overfitting.
- ii) Explain why validation accuracy decreases despite training accuracy increasing.
- iii) Suggest two methods to improve validation performance.



The screenshot shows a Python Online Compiler interface with a code editor on the left and an output window on the right. The code in the editor identifies the epoch where overfitting begins by comparing training and validation accuracy over 10 epochs. The output window displays the results of the code execution.

```
main.py
1 train_acc = [60, 65, 70, 75, 80, 85, 88, 92]
2 val_acc = [58, 63, 68, 72, 73, 72, 70, 68]
3
4 overfit_epoch = None
5
6 for i in range(1, len(val_acc)):
7     if val_acc[i] < val_acc[i-1]:
8         overfit_epoch = i + 1
9         break
10
11 print("Model starts overfitting at Epoch:", overfit_epoch)
12
13 print("\nReason:")
14 print("Training accuracy increases because the model memorizes training data,")
15 print("while validation accuracy decreases due to poor generalization.")
16
17 print("\nMethods to improve validation performance:")
18 print("1. Data Augmentation")
19 print("2. Regularization / Dropout")
```

Output

Model starts overfitting at Epoch: 6

Reason:  
Training accuracy increases because the model memorizes training data,  
while validation accuracy decreases due to poor generalization.

Methods to improve validation performance:  
1. Data Augmentation  
2. Regularization / Dropout

=== Code Execution Successful ===